



SCHOOL OF MATHEMATICS AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : TRINITY KAMBANJE & FRANCIS NYANGU

REGISTRATION NO : BSC-COM-NE-25-23 & BSC-COM-NE-13-21

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION TO
DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : THEORY PAPER

DUE DATE : 8TH MAY, 2026

A. Serialization choices

The telemetry pipeline employs three distinct serialization formats, each chosen for its suitability in context.

- **YAML** is used for configuration because it is highly human-readable and intuitive for non-technical users such as farmers or technicians. Its indentation-based structure makes hierarchical data, like sensor identity and reporting intervals, easy to express. While YAML is more verbose than binary formats, configuration files are read infrequently, so clarity outweighs performance concerns.
- On the sensor-to-server link, **Protobuf (Protocol Buffers)** is selected for efficiency. Sensors generate frequent readings, and Protobuf's compact binary encoding minimizes bandwidth usage while ensuring fast serialization and deserialization. Its schema enforcement guarantees consistent interpretation across devices and supports forward compatibility when new fields are added.
- Finally, the **REST API** supports **JSON and XML/YAML** to accommodate diverse clients. JSON is lightweight and widely supported in browsers and mobile apps, while XML or YAML provide compatibility with legacy desktop tools. Server-driven content negotiation via the `Accept` header ensures flexibility without duplicating logic. In summary, YAML prioritizes readability, Protobuf ensures efficient telemetry transport, and JSON/XML maximize interoperability.

A. Web service protocol

- The historical-data API is implemented using **REST**, which is stateless and resource-oriented. REST maps naturally onto the system's entities: **sensors** and **readings**, and uses familiar HTTP verbs (**GET, POST, PUT, DELETE**) to manipulate them. Its simplicity makes it accessible to heterogeneous clients, from mobile apps to scripts.
- By contrast, **SOAP** is heavy, XML-centric, and requires strict contracts, making it unsuitable for lightweight telemetry pipelines.
- **RPC** focuses on procedures rather than resources, leading to tight coupling and reduced flexibility when APIs evolve.

REST's statelessness ensures scalability, while its resource orientation aligns directly with the data model. For these reasons, REST is the most appropriate protocol for exposing historical sensor data.

B. AsyncIO vs Threading

The telemetry system is fundamentally an **I/O-bound problem**: many sensors send small packets over sockets, and the server spends most of its time waiting for input/output rather than performing heavy computation.

AsyncIO co-routines and the event loop are ideal here, allowing thousands of concurrent connections without the overhead of threads. This model simplifies concurrency, avoids race conditions, and scales efficiently.

Threads or processes are better suited for **CPU-bound tasks**, such as image processing or complex analytics, where parallel execution across cores is beneficial. In this system, however, the bottleneck is network I/O, not computation. Thus, AsyncIO provides the most efficient and maintainable solution for handling concurrent sensor streams.

D. HTTP Mechanics

We have chosen two HTTP features that directly shape the system's design: **content negotiation** and **cookies**.

- **Content negotiation** allows the server to inspect the client's **Accept** header and return data in JSON, XML, or YAML as requested. This ensures compatibility with diverse clients without requiring separate endpoints, making the API flexible and future-proof.
- **Cookies** are used to identify client sessions in the REST API. They enable per-client tracking, enforce session identity, and support access control. By embedding session information in cookies, the server can distinguish between clients even when they share the same endpoints. Together, these mechanics ensure that the system is both interoperable and secure, balancing usability with proper session management.

Both directly shape design: negotiation ensures interoperability, cookies enforce session identity.

REFERENCES

Herlihy, M., & Shavit, N. (2012). *The Art of Multiprocessor Programming*. Morgan Kaufmann

Ben-Kiki, O., Evans, C., & Ingerson, B. (2009). *YAML Ain't Markup Language (YAML™) Specification*. Retrieved from <https://yaml.org/spec/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

W3C. (2007). *SOAP Version 1.2 Part 1: Messaging Framework*. Retrieved from <https://www.w3.org/TR/soap12-part1/> ([w3.org in Bing](#))

Python Software Foundation. (2026). *asyncio — Asynchronous I/O*. In *Python 3 Documentation*. Retrieved from <https://docs.python.org/3/library/asyncio.html> ([docs.python.org in Bing](#))

Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. RFC 7231. Retrieved from <https://www.rfc-editor.org/rfc/rfc7231>

Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455. Retrieved from <https://www.rfc-editor.org/rfc/rfc6455>